Project Group 14 Final Report: Computer Vision Enabled Mobile Garbage Can

Jadon Tsai 1010870283
Dingsheng (Jack) Liu 1009891128
Chuhan Meng 1009868029
Jeffery Tian 1010869945

1 Introduction

This report presents the design and testing of a robotic trash can that moves towards a user based on visual input. The system aims to improve accessibility by reducing the need for users to physically go to a trash can. We use a phone camera attached the front of the robot for perception, which is then sent to an external laptop for position feature processing. These features are transmitted to our onboard microcontroller (STM32 NUCLEO-WB55RG) via Bluetooth Low Energy (BLE), where an onboard PID control algorithms processes the data and sends commands to the motors.

1.1 Initial Plan

Initially, the main idea of this project was a mobile trash can that could autonomously move to intercept garbage before it hit the ground. The system was expected to receive information about the thrown object, calculate the trajectory of its flight, and move to the place where the garbage lands. To accomplish the aforementioned tasks, the project was developed in three stages. The first stage was solely intended to test the creation of a manually controlled platform, equipped with a set of push buttons for directional control. The second stage included creating wireless communication between the laptop and the microcontroller through the use of (BLE) and the UART protocol. The final planned stage involved computer vision, which meant tracking an object by a camera and sending the position data to the platform.

1.2 Design Changes

One of the first issues we encountered was with the motors; we expected them to be able to have a max RPM of about 250, but when we initially tested we found a max RPM of around 120 (we counted this by hand). To address this, we tried to add a gearbox to increase the output speed, but we realized that wasn't a great idea as our motor driver had a somewhat lower current rating compared to the motors.

Our biggest issue came from the motor encoders though. We planned to use them for feedback in our control loop, but when we checked the encoders using an oscilloscope, we found that they were not producing reliable signals. This meant we couldn't really use them for measuring speed or position, which limited our ability to implement our originally intended control loop. Rather than waiting on a new motor, we decided to modify our control approach slightly. Our original control loop required relatively fast response times and accurate motion estimation, which we could not really get with our available hardware. We had to track something slower, so we decided to keep the original trash can idea and track a person instead. By detecting a person and measuring their position relative to the camera frame, the robot could move toward them in a more controlled and predictable way, and have reliable feedback that didn't depend on motor data. As well, as our motors didn't have encoders anyways, we decided to use another set of motors we had that didn't have encoders but ran at a higher RPM to avoid the use of gears. These motors ran at 6V, compared to the 12V of the initial motors, which did cause some issues down the line because our batteries were 7.4 V. Our solution to this is detailed in physical design.

2 Final System Design and Architecture

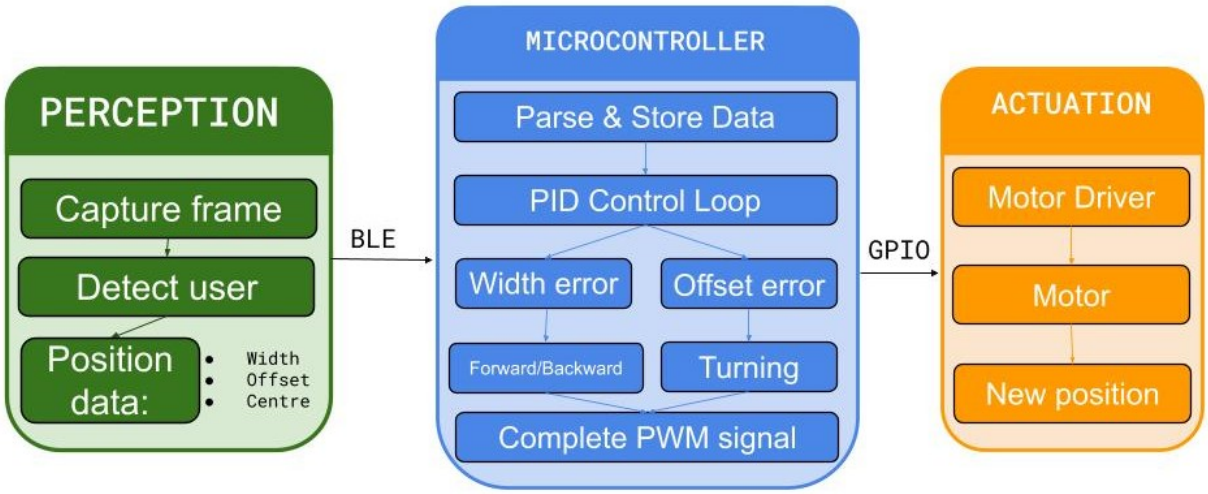

Figure 1: Our system architecture.

2.1 Physical Design

The physical design of the robot was relatively simple. The base of the system consists of a square piece of plywood, where we attached two motor mounts for the motors, and a caster wheel at the front for a third point of contact with the ground. Because the two motors are driven independently, the system uses differential drive so that it can turn. A plastic trash can was then mounted on top of the plywood base by drilling through the bottom and screwing it to the plywood. All hardware except the motors (i.e. the microcontroller, motor driver, power supply, and wiring) were placed inside the trash can, and bulky parts were velcroed down. We did not use permanent attachments as we had to constantly remove and replace parts for battery charging and programming of the microcontroller. All mechanical fabrication and assembly were completed at MyFab. For the computer vision aspect, a phone was mounted to the front of the trash can so that its camera faced outward. The phone was also used for powering the microcontroller since we only had one voltage regulator, which we had to use for the new motor (the intial motors did not need voltage regulators). Placing the phone at the front simplified the interpretation of left-right offset in the camera frame, since it aligned with the robot's direction of motion.

2.2 Electrical Design

The circuit is built around the STM32 NUCLEO-WB55RG [1] microcontroller, which handles all control logic and also has an built in BLE communication module. We use a motor driver (TB6612FNG) [2]that has two channels, and each controls one of two DC motors (25GA370) [3]. The microcontroller is powered by the phone connected through the micro USB port, and the motors are powered from a 7.4V LiPo battery stepped down to 5V using a 5V voltage regulator.

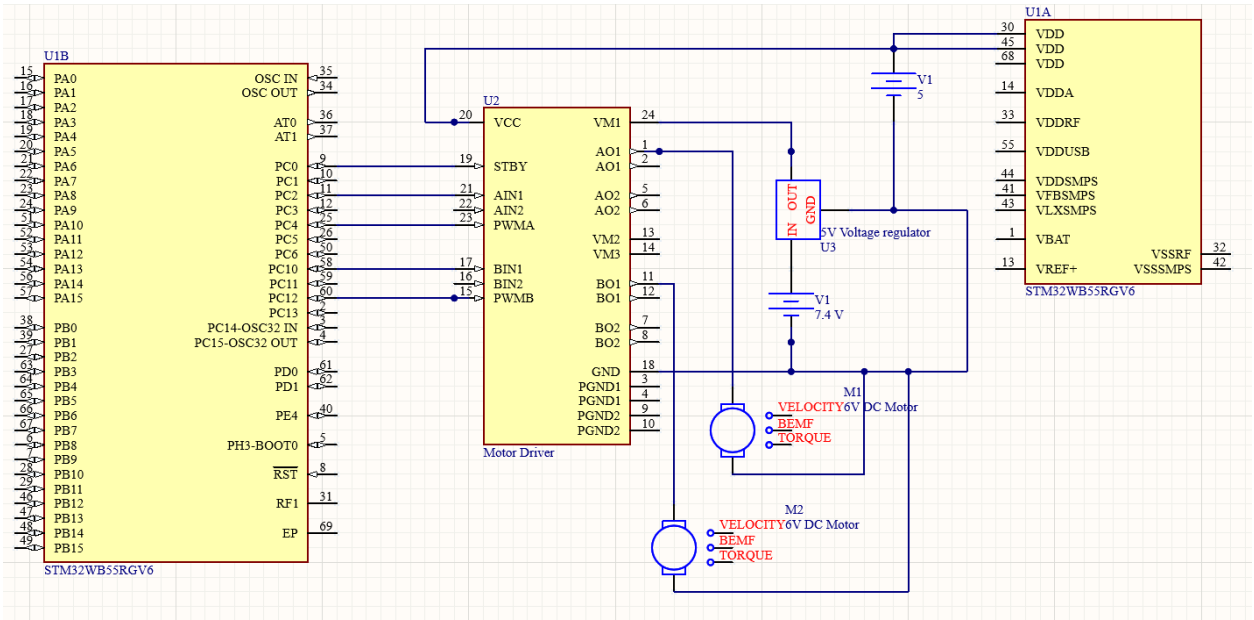

Figure 2: Our circuit design.

2.3 Embedded Firmware Design

The embedded firmware is designed to enable real-time human tracking using a vision-based control system. The system interacts with the user indirectly through visual input, where a phone camera detects the user and extracts key features of the bounding box surrounding them. Specifically, the vision system computes three values: the width of the bounding box (representing distance to the person), the pixel offset of the bounding box center from the image center (representing horizontal error), and the absolute center position of the bounding box. These values are transmitted via BLE to the microcontroller in the form of a comma-separated string.

Upon receiving this data, the firmware parses and stores the values, which are then used in a fixed-rate control loop running on the microcontroller. The control logic is based on a proportional–integral–derivative (PID) framework that governs the robot's motion. The width of the bounding box is used as the primary input for forward and backward motion, allowing the robot to maintain a desired distance from the user, while the horizontal offset from the image centre is used to adjust lateral motion, enabling the robot to align itself with the person. The control stage executes the PID loop at a fixed interval, computes motor commands, and updates pulse-width modulation (PWM) signals to drive the motors accordingly. By combining these two control objectives, the robot continuously follows and centres itself relative to the user.

3 Microcontroller Features and Resource Usage

To control the motors, we mainly used the GPIO pins and hardware timers present on the microcontroller. The motor driver requires two GPIO inputs per motor to control the motor direction, as well as two PWM input pins to control the speed of each motor, for a total of four GPIO outputs and two hardware timers channels generating a PWM output. We also used the microcontroller's BLE capabilities to communicate with a laptop, which gave us the inputs for our PID control.

We made great use of the GPIO and hardware timer PWM functionality, as we constantly had to change the GPIO output and PWM duty cycle depending on our PID control output. However, there were certain aspects, like GPIO interrupts and hardware timer features like input capture, of both features that we did not use. Additionally, there were other features on the microcontroller, such as its security features that we did not use, and we made relatively limited use of the full capabilities of the BLE in that context.

To improve this project, we could have used other types of sensors, particularly motor encoders. This was part of our original plan, we wanted to use the hardware timer's input capture or time tracking features in conjunction with GPIO interrupts to track the instantaneous speed, and thus the relative position of the robot.

4 Software Tools, Libraries, and Design Methods

The Python portion of the project was implemented using a combination of OpenCV [4] and a deep learning–based object detection model (YOLO) [5] to perform real-time human detection from a phone camera feed. The person is only detected in the frame if the computer vision model (YOLOv8) has a confidence higher than 0.4. Each video frame is processed to detect a person and extract key features of the bounding box, including its width and horizontal offset from the frame centre. These values serve as inputs to the control system and are formatted into a simple comma-separated string for transmission over BLE. For simplicity and reliability, the design follows a modular and loop-based structure, where image acquisition, detection, feature extraction, and communication are executed sequentially within a continuous processing loop. In retrospect, while this design is effective, it would be improved by introducing a more explicit state machine (e.g. detection vs. no-detection states). The code implementation can be found in `python_cv_code/garbage_bin_yolo.py`.

On the C side, we made heavy use of a toolchain consisting of STM32CubeMX to configure the STM32 and generate the relevant IOC file and initialization code, the STM32CubeIDE VSCode extension for actual programming and building via CMake, and the STM32CubeProgrammer to flash the built ELF file onto the STM32. From the STM32CubeMX, we used the P2P server example available for our NucleoWB55RG as the backbone of our BLE communication software and made very few modifications, save for the actions performed when the P2P server received a message from the client it was connected to. However, we did not see the need to use any other libraries, both for our PID control and for our motor output, as we felt we would be able to write all the necessary code ourselves. We made heavy use of a PID control loop, and it formed the backbone of our motor control, but we also employed noise-reduction concepts like hysteresis (see the deadband functions) to make the robot run smoother and prevent unnecessary power draw once the PWM signal to the motors fell below a specific threshold value and the motors did not have enough power to move the robot anymore. However, we were unable to make extensive use of interrupts or communication protocols discussed in class (I2C, SPI, CAN), as our design didn't have any sensors that used the aforementioned protocols, and though we had the opportunity to use GPIO interrupts to track the rotation of our motors via their encoders, we quickly found that our encoders were either faulty or dead, locking us out of any options to use either. Specific code logic can be found in `embedded_code/src/main.c, embedded_code/src/pid.c` and BLE configuration can be found within `embedded_code/STM32_WPAN` folder.

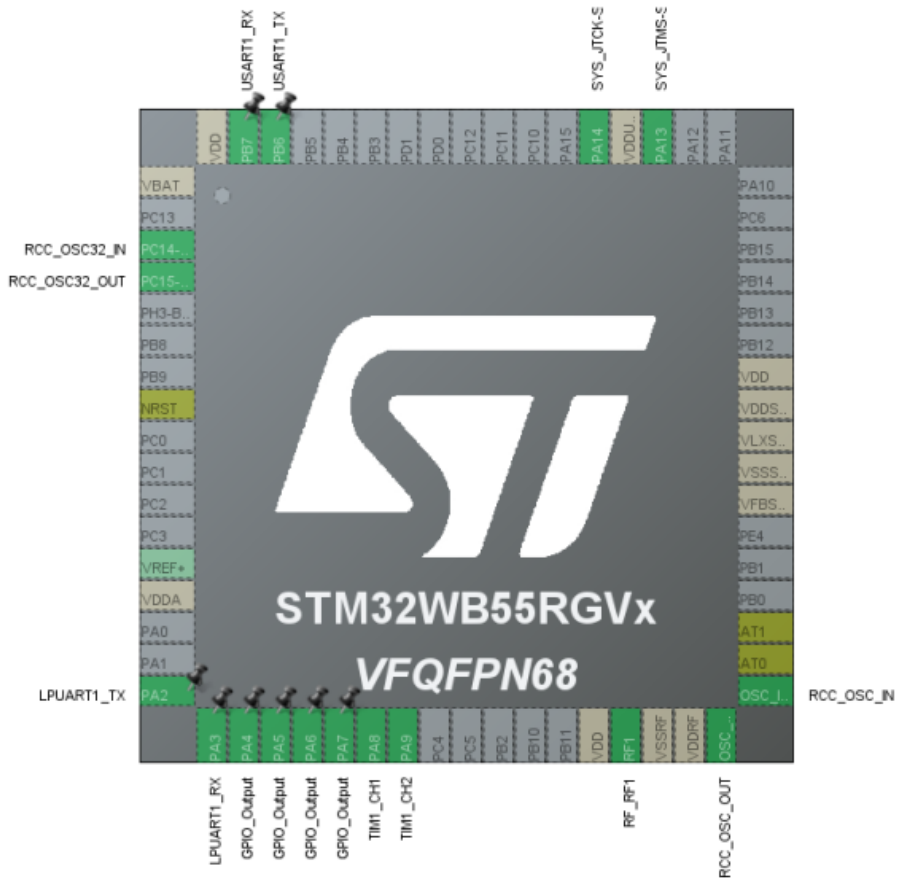

Figure 3: The ioc configuration for our microcontroller

5 Testing, Results, and System Validation

For system validation, two types of tests were conducted in a standard classroom. The main goal for the robot was to detect a person through the camera and then move autonomously towards the person, so we performed two tests: one with a stationary target and one with a moving target. The metric for the stationary test was the distance between the robot and the user at the end of the robot's movement. A trial was regarded as successful if the robot managed to stop within 0.5 m of the user, which is approximately an arm's length, from the person and reaches the person within 5 seconds with a starting point across the room (roughly 5 metres). The task success rate is determined as the ratio of successful trials to all the performed trials. In the testing, the system has been tested 20 times, out of which only two were considered unsuccessful, thus giving a success ratio 90%.We also wanted to track the ability of the robot to follow a moving target. The user walked around in a circle, and we qualified a success if the robot was able to track, and also follow, the user through the entire circle. It succeeded in this test eight out of ten attempts, thus showing an 80% success rate. From these results, we can say that the robot can track the moving target, though with less accuracy than tracking a stationary target.

5.1 Design Changes Due to Testing

During testing, several adjustments were made to improve system performance. The PID controller was tuned by increasing the derivative gain to improve turning response and reduce overshoot. The object detection confidence threshold was lowered to improve detection reliability. The stopping condition was also modified by adjusting the bounding box width threshold, creating a larger deadband so the robot would stop at an appropriate distance from the person instead of oscillating around the target.

5.2 Limitations and Future Improvements

The system had several limitations. The motor power was limited by the fact that we had to repurpose a voltage regulator, so we could only get 5V rather than the full 6V (we did order a buck converter but we blew that up by accident). This reduced the robot's speed and acceleration and affected response time. Future improvements could focus on strengthening sensing and control. Incorporating motor encoders with interrupts and timer input capture would enable fully closed-loop feedback, allowing more accurate velocity and position control instead of relying solely on computer vision data that is more bulky. This would significantly improve tracking stability and response. Additionally, integrating sensors (e.g. IMUs or distance sensors) using protocols we learned in class, such as I2C or SPI, would provide more robust state information.

6 Connections to Course Content

For our communication, although BLE and UART were not covered directly, the way we structured and sent data between the laptop and the microcontroller follows the same ideas as the communication protocols from class. We had to package position data, send it at a predictable rate, and then finally read it on the microcontroller. During the debugging process, we encountered many common embedded code issues that were taught in class. For example, we had issues with unsigned and signed integer data parsing, BLE packet overflow and PID control overflow (specifically for the integral and derivative terms). For motor control, we used PWM signals from the microcontrollers in built PWM enabled pins, which directly applies what we learned about PWM in class. Our robot's motion is controlled using a PID loop based on the position error from the camera. We tuned the parameters through testing to get reasonable behavior without excessive oscillation or delay. This is similar to what we did in lab 9, and we also added a simple hysteresis band, similar to what we learned in class.

References

[1] STMicroelectronics. Stm32 nucleo-wb55rg development board. https://www.st.com/en/evaluation-tools/nucleo-wb55rg.html, 2023. Online; accessed 2026-04-07.

[2] Toshiba Electronic Devices & Storage Corporation. Tb6612fng dual dc motor driver ic datasheet. https://toshiba.semicon-storage.com/info/docget.jsp?did=10660, 2017. Online; accessed 2026-04-07.

[3] 25ga370 dc gear motor specifications. https://www.pololu.com/product/1103. Datasheet; Online; accessed 2026-04-07.

[4] OpenCV. Open source computer vision library. 2024. Accessed: April 7, 2026.

[5] Glenn Jocher et al. Ultralytics yolov8. https://github.com/ultralytics/ultralytics, 2023. Accessed: 2026-04-07.

7 Appendices

7.1 Video links

Our video is available here. Our oscilloscope testing is available here.

7.2 Code link

Our repository is available here, and we have also attached a zip file of our code.

7.3 Physical picture